

# astrid-runtime/astrid

Astrid : un runtime Rust capability-secure pour capsules WebAssembly, porté par un seul mainteneur

Astrid is a portable, capability-secure operating system for composable software.

10277 ★

137 forks

216 issues ouvertes

Apache-2.0

<https://github.com/astrid-runtime/astrid>

## Que fait ce projet et pour qui ?

---

Astrid se décrit comme « a portable, capability-secure operating system for composable software »<sup>1</sup> : aujourd'hui un runtime user-space sur macOS et Linux, la cible affichée étant un OS autonome<sup>2</sup>. Les composants sont des **capsules** WebAssembly exécutées dans Wasmtime sans autorité ambiante ; le noyau ( `astrid-daemon` ) se contente de router des événements IPC sous ACL de capacités, sans handle LLM ni registre d'outils<sup>2</sup>. Public visé : qui construit des systèmes d'agents, des outils, des services ou une distribution, et refuse que le modèle soit l'autorité qui autorise ses propres actions ; Astrid n'impose ni produit, ni fournisseur de modèle, ni boucle d'agent<sup>2</sup>. Topics cohérents : `rust`<sup>3</sup>, `wasm`<sup>4</sup>, `mcp`<sup>5</sup>, `capability-security`<sup>6</sup>. Quatre binaires : `astrid` (CLI uplink, TUI et mode headless), `astrid-daemon` (noyau), `astrid-build` (compile les capsules vers `wasm32-unknown-unknown`), `astrid-emit` (pont stdio → bus)<sup>2</sup>. Chiffres : 10277 étoiles<sup>7</sup>, 137 forks<sup>8</sup>, 216 issues ouvertes<sup>9</sup>, dépôt créé le 2026-02-15<sup>10</sup> par une organisation<sup>11</sup>, dernier push le 2026-09-09<sup>12</sup>. Licence relevée par GitHub : Apache-2.0<sup>13</sup> ; le README et les fichiers `LICENSE-APACHE` / `LICENSE-MIT` disent « MIT OR Apache-2.0 »<sup>2 14 15</sup>. Le cache ne contient pas la répartition des langages (bloc `languages` absent) ; à vérifier sur <https://github.com/astirid-runtime/astirid>. Le manifeste est un workspace Cargo<sup>16</sup>.

# Comment le code est-il organisé ?

---

`.cargo/` config Cargo locale

---

`.dockerignore` exclusions du build Docker

---

`.gemini/` config agent Gemini

---

`.gitattributes` attributs git

---

`.github/` 20 workflows GitHub Actions

---

`.gitignore` exclusions git

---

`.gitmodules` sous-modules git

---

`CHANGELOG.md` journal des versions, 410 Ko

---

`CONTRIBUTING.md` guide de contribution

---

`Cargo.lock` verrou des dépendances Rust

---

`Cargo.toml` manifeste du workspace Rust

---

`LICENSE-APACHE` licence Apache 2.0

---

`LICENSE-MIT` licence MIT

---

`README.md` présentation du projet

---

`SECURITY.md` politique de sécurité

---

`changes/` notes de changement en attente

---

`clippy.toml` config lint clippy

---

`container/` définitions de conteneur

---

`crates/` code principal : 33 crates Rust, 32 membres du workspace

`docs/` documentation

`e2e/` scénarios TOML et fixtures bout en bout

`fuzz/` tests de fuzzing

`native/` code natif macOS

`release/` config de release (nightly.toml)

`rust-toolchain.toml` version Rust épinglée

`rustfmt.toml` config formatage rustfmt

`scripts/` scripts build, CI, release

`wit` interfaces WIT, entrée de 0 octet

Workspace Cargo ( `Cargo.toml` , resolver 2, 32 membres) <sup>17</sup> <sup>18</sup> ; tout le code est sous `crates/` : 33 crates, dont `astrid-cli-mockup` hors workspace <sup>19</sup> <sup>20</sup> . Repères par rôle, d'après les noms de crates <sup>20</sup> : noyau et bus ( `astrid-kernel` , `astrid-daemon` , `astrid-runtime` , `astrid-events` , `astrid-hooks` , `astrid-config` ); sécurité ( `astrid-crypto` , `astrid-capabilities` , `astrid-audit` , `astrid-approval` , `astrid-vfs` , `astrid-core` ); état ( `astrid-storage` , `astrid-storage-provider-fuse` / `-fskit` / `-winfsp` , `astrid-storage-chunker-evidence` ); frontends ( `astrid-cli` , `astrid-gateway` , `astrid-mcp` , `astrid-uplink` ); outillage capsules ( `astrid-build` , `astrid-capsule` , `astrid-capsule-install` , `astrid-capsule-types` , `astrid-emit` ). Frontière de sécurité, réservée aux tiers Core et Maintenir : `astrid-crypto` , `astrid-capabilities` , `astrid-audit` , `astrid-approval` , `astrid-vfs` , `astrid-storage` , `astrid-sys` , `astrid-core` <sup>21</sup> . Note : `astrid-sys` n'apparaît pas dans le `crates/` relevé <sup>20</sup> . Autour du code : `e2e/` (scénarios TOML et fixtures bout en bout) <sup>22</sup> , `fuzz/` <sup>23</sup> , `crates/``astrid-integration-tests` (39 fichiers, cargo test + insta) <sup>24</sup> <sup>25</sup> <sup>26</sup> , `native/macos/` <sup>27</sup> , `docs/` <sup>28</sup> , `scripts/` (build, CI, release) <sup>29</sup> , `changes/` (fragments de changelog en attente) <sup>30</sup> , `.github/` avec 20 workflows <sup>31</sup> . Deux curiosités : `wit` est un fichier de 0 octet à la racine <sup>32</sup> et `.gitmodules` déclare des sous-modules <sup>33</sup> ; le cache ne dit pas ce qu'ils pointent, à vérifier dans <https://github.com/strid-runtime/strid/blob/main/.gitmodules> . Aucun repère pour agents IA (AGENTS.md, CLAUDE.md, .cursorrules...) : score 0 sur 7 <sup>34</sup> <sup>35</sup> ; seul un dossier `.gemini/` existe <sup>36</sup> .

## Par où commencer à lire ?

---

1. `crates/astrid-cli/src/main.rs` <sup>37</sup> : binaire `astrid`, `#[tokio::main]` qui parse clap puis route vers `dispatch` ; client léger qui se connecte au kernel (auto-démarrage), crée ou reprend des sessions et rend les événements en streaming <sup>38</sup>. 2. `crates/astrid-cli/src/cli.rs` <sup>39</sup> : définitions clap des sous-commandes (`agent`, `capsule`, `quota`, `status`, `start`, `stop`, `ps`, `top`) ; déduit du `mod cli` de `main.rs`, fichier non lu par le collecteur <sup>40</sup>. 3. `crates/astrid-daemon/src/main.rs` <sup>41</sup> : 26 lignes, `setuid` puis runtime tokio multi-thread qui appelle `astrid_daemon::run()` <sup>42</sup>. C'est la porte vers le noyau. 4. `crates/astrid-cli-mockup/src/main.rs` <sup>43</sup> : maquette de CLI hors workspace, à ignorer au début <sup>44</sup>. Pour le modèle mental, lire d'abord les sections « How it works » et « The security model » du README (`uplinks`, `bus`, `Capsule.toml` avec `[imports]` / `[exports]`, ABI WIT `astrid:*`) <sup>2</sup>, puis The Astrid Book, présenté comme la référence canonique avec ancrés fichier/ligne <sup>45</sup>. Le SDK des capsules (`astrid-sdk`) vit dans un autre dépôt <sup>46</sup>. Le cache ne contient pas le corps de `astrid_daemon::run()` ni `crates/astrid-kernel/src/lib.rs` ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/crates/astrid-daemon/src> et <https://github.com/astrid-runtime/astrid/tree/main/crates/astrid-kernel/src>.

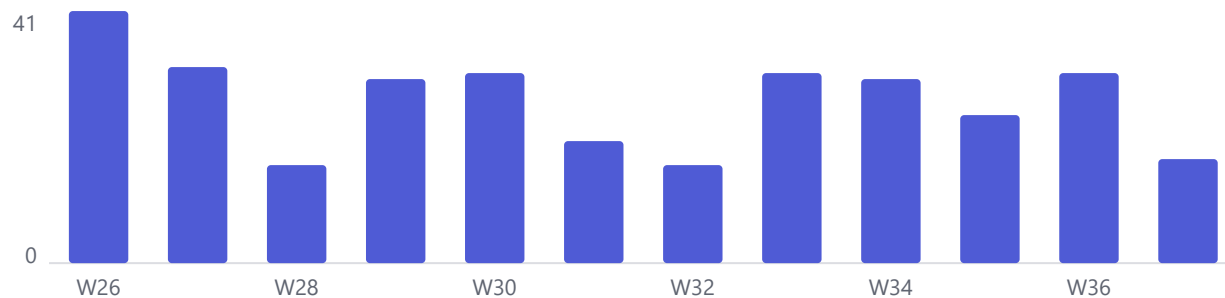
## Comment builder, tester et lancer ?

---

Rust 1.95 minimum, édition 2024 <sup>18</sup>, version épinglée par `rust-toolchain.toml` à la racine <sup>47</sup>. Depuis un shell : ``` bash git clone https://github.com/astrid-runtime/astrid cd astrid && cargo build --release # binary at ./target/release/astrid cargo test --workspace -- --quiet astrid start && astrid status && astrid stop ``` Commandes prises dans <sup>48</sup>, <sup>49</sup>, <sup>50</sup> et le « Quick start » du README <sup>2</sup>. Alternatives : `brew tap astrid-runtime/tap && brew install astrid`, ou `cargo install astrid` <sup>2</sup>. Avant une PR, rejouer la CI en local : `cargo fmt --all -- --check`, `cargo clippy --workspace --all-features --all-targets --locked -- -D warnings`, `cargo test --workspace --locked` <sup>51</sup> ; CONTRIBUTING exige au minimum `cargo test --workspace` et `cargo clippy -- -D warnings` <sup>21</sup>. Lints du workspace : `unsafe_code = "deny"`, `clippy pedantic` en warn, `arithmetic_side_effects = "deny"` <sup>18</sup>. CI : `.github/workflows/ci.yml` <sup>52</sup>, déclenchée sur `push` <sup>53</sup> (branches `main` et `stream-*`) et `pull_request` <sup>54</sup> ; 14 jobs dont `check`, `fmt`, `clippy`, `test` (ubuntu et macos), `msrv 1.95`, `cargo-audit`, portabilité `wasm32`, tests natifs Windows <sup>51</sup> <sup>55</sup>. 20 workflows au total <sup>31</sup>. Le cache ne contient pas les prérequis système (FUSE, FSKit, WinFsp) des tests de stockage ni le temps de build ; à vérifier dans `scripts/` <sup>29</sup> et <https://github.com/astrid-runtime/astrid/blob/main/.github/workflows/ci.yml>.

## Comment le projet vit-il ?

---



Activité soutenue : entre 16<sup>56</sup> et 41<sup>57</sup> commits par semaine sur les 12 dernières semaines (2026-W26<sup>58</sup> à 2026-W37<sup>59</sup>), dernier commit le 2026-09-09T13:46:34Z<sup>60</sup>. Mais une seule personne porte le projet : joshuaJBouw 309 commits<sup>61</sup>, jvsteiner 7<sup>62</sup>, Copilot 3<sup>63</sup>. Bus factor 1<sup>64</sup>, risque noté high<sup>65</sup> : « Un seul contributeur porte la moitié des commits sur 12 semaines »<sup>66</sup>. Conséquence pratique : un seul relecteur pour vos PR ; 24 PR ouvertes<sup>67</sup> pour 188 fusionnées en 30 jours<sup>68</sup>. Le cache ne dit pas combien attendent une revue (le serveur MCP n'expose pas les reviews) ; à vérifier dans <https://github.com/astroid-runtime/astroid/pulls>. Releases : v2026.9.0 publiée le 2026-09-09<sup>69 70</sup>, première du schéma year.month.patch annoncé par le README<sup>2</sup> ; avant elle, de v0.9.3 (2026-07-06)<sup>71 72</sup> à v0.10.4 (2026-07-20)<sup>73 74</sup>, plus des tags de canal channel-stable, channel-nightly, channel-dev<sup>75 76 77</sup>. Dépôt créé le 2026-02-15<sup>10</sup>, 10277 étoiles<sup>7</sup> pour 137 forks<sup>8</sup> : beaucoup de regards, peu de mains. Le cache ne contient ni l'historique avant W26 ni un rythme de release annoncé ; à vérifier dans CHANGELOG.md (410 Ko)<sup>78</sup> et <https://github.com/astroid-runtime/astroid/releases>.

## Quelle première contribution ?

---

Processus strict : issue d'abord, attendre l'assignation par un mainteneur, label `newcomer-approved` posé sur la PR d'un nouveau, closes #N, template PR entièrement rempli, commit signé `git commit -s` (DCO), fragment `changes/{issue}.{kind}.md`, titre Conventional Commits ; une PR sans issue liée est fermée <sup>21</sup>. 5 issues `good first issue` <sup>79</sup> sur 192 ouvertes <sup>80</sup> (149 fermées en 30 jours <sup>81</sup>, 42 étiquetées `bug` <sup>82</sup>) : - #459 « Markdown parser in TUI fails on numbered lists  $\geq 10$  » <sup>83 84</sup> : la TUI est dans le binaire `astrid` <sup>2</sup>, donc `crates/astrid-cli`, crate non-core ouverte aux nouveaux <sup>21</sup> ; périmètre borné, test de régression évident : meilleur candidat. - #276 « MCP interceptor: propagate `tool_result.is_error` instead of treating it as success » <sup>85 86</sup> : probablement `crates/astrid-mcp`, hors liste des crates de sécurité <sup>21</sup>. - #497 « Remove dead `AgentHandler` stub from `astrid-hooks` » <sup>87 88</sup> : `astrid-hooks` est un crate core, à confirmer avec le mainteneur avant de la réclamer <sup>21</sup>. Restent #476 <sup>89</sup> et #461 <sup>90</sup>. Repère de style : les PR fusionnées suivent `fix(scope): ... / feat(scope): ...`, ex. « `fix(init): honor explicit grants after completed batch resume` » fusionnée le 2026-09-09 <sup>91 92</sup>. Éviter au début les fils chauds comme #1708 (29 commentaires) <sup>93</sup>. Le cache ne dit pas si ces issues sont déjà assignées ; à vérifier sur chaque URL avant de commenter pour la réclamer.



## Vos 3 premières actions

---

1. Installer : `git clone https://github.com/astrid-runtime/astrid cd astrid && cargo build --release # binary at ./target/release/astrid`
2. Lancer les tests : `cargo test --workspace -- --quiet`
3. Ouvrir `crates/astrid-cli/src/main.rs` — binaire ``astrid`` : `#[tokio::main]` qui parse clap (`cli::Cli`) puis route vers dispatch ; client léger qui se connecte au kernel (auto-démarrage), crée ou reprend des sessions et rend les événements en streaming. Confirmé (en-tête du fichier).

# Sources

---

1. donnée collectée `repo.description`
2. source du repo `readme.md`
3. donnée collectée `repo.topics.6`
4. donnée collectée `repo.topics.8`
5. donnée collectée `repo.topics.4`
6. donnée collectée `repo.topics.2`
7. donnée collectée `repo.stars`
8. donnée collectée `repo.forks`
9. donnée collectée `repo.open_issues`
10. donnée collectée `repo.created_at`
11. donnée collectée `repo.owner_type`
12. donnée collectée `repo.pushed_at`
13. donnée collectée `repo.license`
14. donnée collectée `tree.11.path`
15. donnée collectée `tree.12.path`
16. donnée collectée `deps.manifest`
17. donnée collectée `tree.10.role`
18. source du repo `manifest.md`
19. donnée collectée `tree.18.role`

20. source du repo `tree.md`
21. source du repo `contributing.md`
22. donnée collectée `tree.20.role`
23. donnée collectée `tree.21.role`
24. donnée collectée `tests.dir`
25. donnée collectée `tests.files`
26. donnée collectée `tests.framework`
27. donnée collectée `tree.22.role`
28. donnée collectée `tree.19.role`
29. donnée collectée `tree.26.role`
30. donnée collectée `tree.15.role`
31. donnée collectée `tree.4.role`
32. donnée collectée `tree.27.role`
33. donnée collectée `tree.6.role`
34. donnée collectée `ai_docs.score`
35. donnée collectée `ai_docs.max`
36. donnée collectée `tree.2.role`
37. donnée collectée `entrypoints.0.path`
38. donnée collectée `entrypoints.0.why`
39. donnée collectée `entrypoints.2.path`
40. donnée collectée `entrypoints.2.why`

- 41. donnée collectée `entrypoints.1.path`
- 42. donnée collectée `entrypoints.1.why`
- 43. donnée collectée `entrypoints.3.path`
- 44. donnée collectée `entrypoints.3.why`
- 45. GitHub <https://github.com/astrid-runtime/book>
- 46. GitHub <https://github.com/astrid-runtime/sdk-rust>
- 47. donnée collectée `tree.24.role`
- 48. donnée collectée `build.install`
- 49. donnée collectée `build.test`
- 50. donnée collectée `build.run`
- 51. source du repo `ci.md`
- 52. donnée collectée `build.ci.0.path`
- 53. donnée collectée `build.ci.0.triggers.0`
- 54. donnée collectée `build.ci.0.triggers.1`
- 55. donnée collectée `risks.3.note`
- 56. donnée collectée `activity.commits_per_week.2.count`
- 57. donnée collectée `activity.commits_per_week.0.count`
- 58. donnée collectée `activity.commits_per_week.0.week`
- 59. donnée collectée `activity.commits_per_week.11.week`
- 60. donnée collectée `activity.last_commit`
- 61. donnée collectée `activity.contributors.0.commits`

62. donnée collectée `activity.contributors.1.commits`

63. donnée collectée `activity.contributors.2.commits`

64. donnée collectée `activity.bus_factor`

65. donnée collectée `risks.1.level`

66. donnée collectée `risks.1.note`

67. donnée collectée `pulls.open`

68. donnée collectée `pulls.merged_30d`

69. donnée collectée `releases.0.tag`

70. donnée collectée `releases.0.date`

71. donnée collectée `releases.9.tag`

72. donnée collectée `releases.9.date`

73. donnée collectée `releases.1.tag`

74. donnée collectée `releases.1.date`

75. donnée collectée `releases.5.tag`

76. donnée collectée `releases.6.tag`

77. donnée collectée `releases.7.tag`

78. donnée collectée `tree.7.role`

79. donnée collectée `issues.good_first`

80. donnée collectée `issues.open`

81. donnée collectée `issues.closed_30d`

82. donnée collectée `issues.by_label.bug`

- 83. donnée collectée `issues.good_first.3.title`
- 84. GitHub <https://github.com/astrid-runtime/astrid/issues/459>
- 85. donnée collectée `issues.good_first.4.title`
- 86. GitHub <https://github.com/astrid-runtime/astrid/issues/276>
- 87. donnée collectée `issues.good_first.0.title`
- 88. GitHub <https://github.com/astrid-runtime/astrid/issues/497>
- 89. donnée collectée `issues.good_first.1.number`
- 90. donnée collectée `issues.good_first.2.number`
- 91. donnée collectée `pulls.merged_30d.0.title`
- 92. donnée collectée `pulls.merged_30d.0.merged_at`
- 93. donnée collectée `issues.hot.0.comments`